Booking Platform Domain Exercise

Flows, design, and PowerShell-friendly curl commands for the scenarios in Version 1.3.3 of the attached exercise.

Code Scope Covered

B2B theatre onboarding, show creation, seat allocation, B2C browse, movie creation/filtering, booking, pricing, cache, and scenario-focused tests.

Primary Runtime

Spring Boot 4, Java 21, PostgreSQL, Ehcache via JCache, Docker Compose for local services.

Assumption for curl sequences: the local database is clean. If your environment already contains data, use the `id` values returned by each create API instead of assuming `1`, `2`, or `3`.

1. Requirement Mapping

Exercise Area	Requirement	Coverage in Current Solution
Read scenario	Browse theatres currently running the show in the town by chosen date.	`GET /browse/shows?movieId=&city=&date=`
Pricing rule	50% discount on the third ticket in selected cities and theatres.	Implemented through `PricingEngine` plus configurable `booking.pricing` rules.
Pricing rule	20% discount for afternoon shows.	Applied for shows from `12:00` up to but not including `16:00`.
Write scenario	Book movie tickets by theatre, timing, and preferred seats.	`POST /bookings` with seat validation and persisted booking.
Write scenario	Theatres can create shows for the day.	`POST /partners/shows`
Write scenario	Theatres can allocate seat inventory for the show.	`POST /partners/shows/{showId}/seats`
B2B onboarding	Theatre partners onboard theatres over the platform.	`POST /partners/theatres`
B2C catalog	Customers access movies by city, language, and genre.	Movie metadata contains `language`; listing supports `genre` and `language` filters.

2. Solution Overview

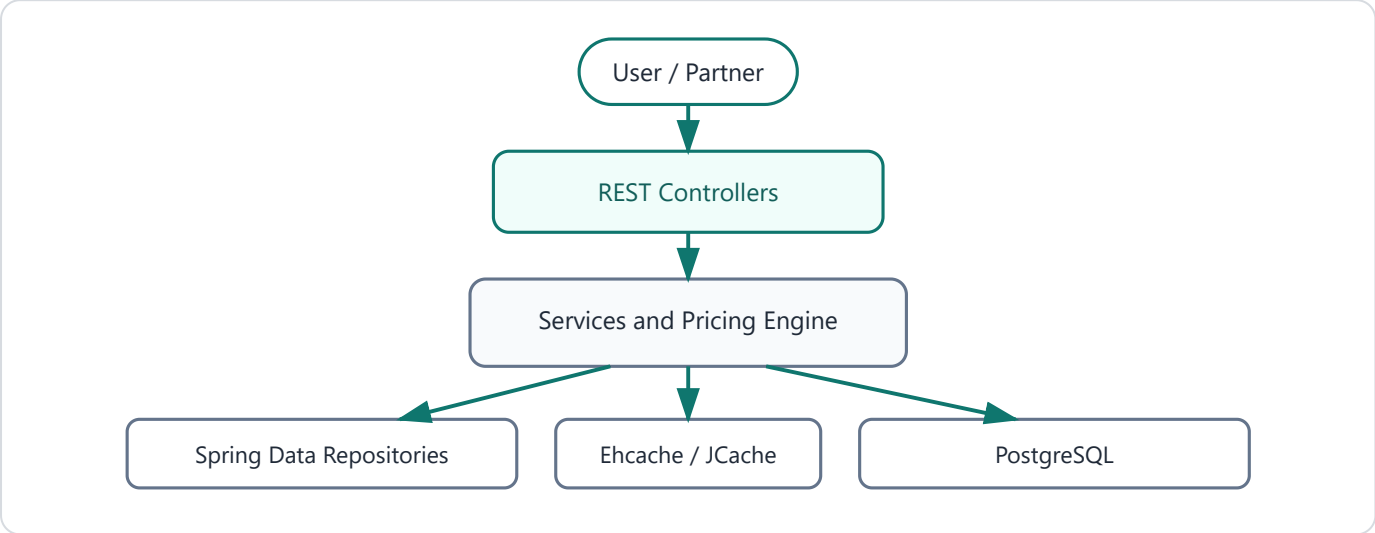
The platform is split into two operating surfaces:

B2B Theatre Partner APIs | B2C Customer Browse APIs | B2C Customer Booking APIs | Config-Driven Pricing | Read-Heavy Cache

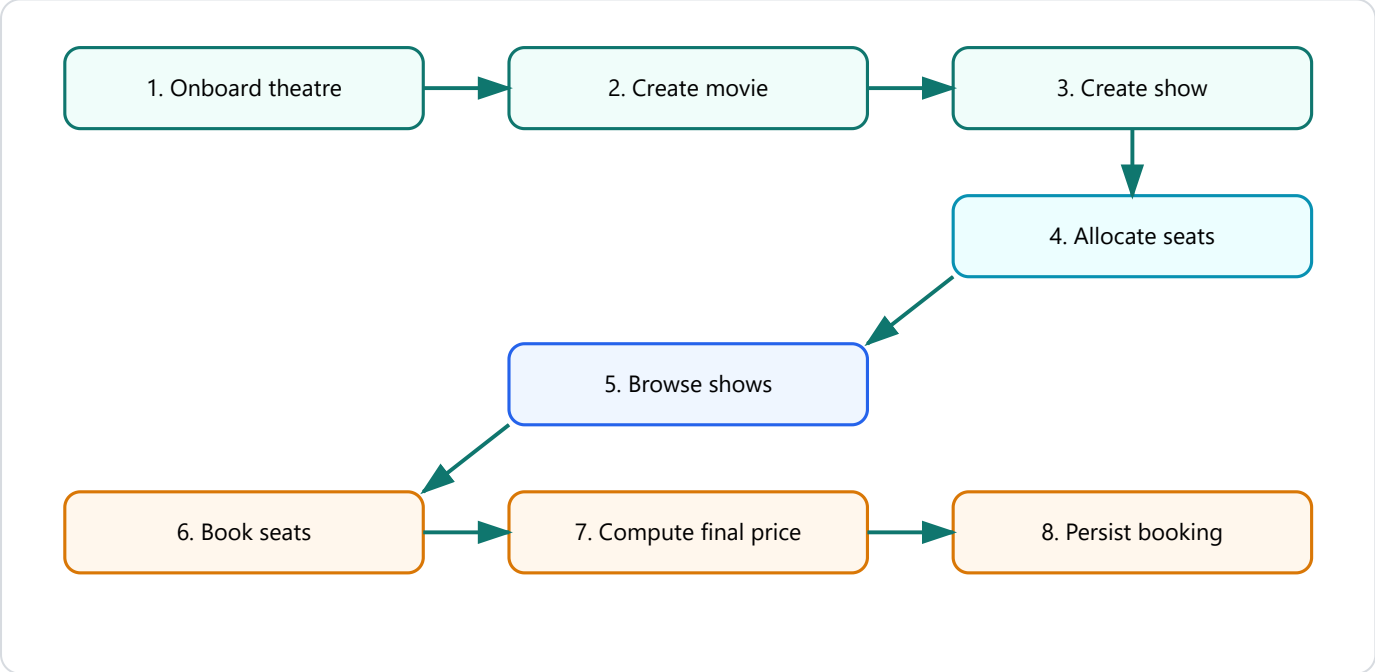
> The current implementation is intentionally optimized for the exercise's most important artifacts: API contract, service implementation, design reasoning, data model, and an end-to-end scenario demonstration without a UI.

3. Architecture and Core Flows

3.1 High-Level Architecture



3.2 End-to-End Business Flow



3.3 Booking Service Flow

1. Receive `showId` and requested seat numbers.
2. Load the `Show` from the repository.
3. Load all requested seats for that show.
4. Validate that all seats exist, are unique, and are not already booked.
5. Mark seats as booked inside a transaction.
6. Compute price using location-based third-ticket logic and afternoon discount logic.

7. Persist the final `Booking` with timestamp and seat relations.

4. Data Model and API Surface

4.1 Core Entities

Entity	Purpose	Important Fields
City	Location grouping for theatres.	`id`, `name`
Theatre	Partner-owned venue in a city.	`id`, `name`, `city`
Movie	Catalog metadata exposed to customers.	`id`, `title`, `genre`, `language`
Show	Movie schedule for a theatre and date/time.	`id`, `movie`, `theatre`, `showDate`, `showTime`, `price`
Seat	Allocatable seat inventory per show.	`id`, `seatNumber`, `show`, `isBooked`
Booking	Customer reservation with computed total.	`id`, `show`, `seats`, `totalPrice`, `createdAt`

4.2 Implemented Endpoints

Method	Path	Purpose
POST	`/partners/theatres`	Onboard theatre partner and city mapping.
POST	`/partners/shows`	Create a show for a movie and theatre.
POST	`/partners/shows/{showId}/seats`	Allocate seat inventory for a show.
POST	`/movies`	Create movie metadata including language.
GET	`/movies`	Filter movies by genre and language.
GET	`/browse/shows`	Read scenario: browse shows by movie, city, and date.
POST	`/bookings`	Write scenario: book selected seats for a show.
GET	`/bookings/{id}`	Fetch persisted booking details.

5. Design Decisions and Non-Functional View

5.1 Transactional Decisions

- `BookingService#bookTickets` runs in a transaction to keep seat marking and booking persistence consistent.
- `TheatreService#onboardTheatre` and `ShowService` seat allocation are transactional to keep parent-child persistence aligned.
- Validation is performed before final save to reduce partial writes.

5.2 Cache Design

- Ehcache via JCache is used for lightweight in-process read caching.
- Current code caches movie lookup and keeps Redis as an optional future scale-out path.
- This choice fits the current single-service deployment model from the exercise implementation.

5.3 Existing Theatre IT Integration

- The partner APIs act as a simple façade for theatre onboarding, show publishing, and seat inventory submission.
- In an extended design, these APIs can be backed by integration adapters for partner PMS or ERP systems.
- For higher throughput, partner updates can be accepted synchronously and published to downstream topics asynchronously.

5.4 Scalability and Availability Direction

- Run multiple stateless Spring Boot instances behind a load balancer.
- Move from local cache to Redis or another distributed cache if horizontal scaling requires shared cache state.
- Use PostgreSQL high availability, read replicas, and multi-AZ deployment for 99.99% availability goals.
- Shard or partition by city/country when traffic or catalog volume grows significantly.

5.5 Security and Compliance

- Input validation and exception handling are in place for the current scope, but authentication and authorization should be added for production.
- Recommended next steps: JWT or OAuth2 for partner/customer identity, rate limiting, audit logs, secrets management, and dependency scanning.
- Payment integration should isolate PCI-sensitive boundaries to a dedicated payment service and gateway redirect/token model.

5.6 Monitoring and KPIs

Area	Suggested KPI
Browse performance	p95 response time, cache hit ratio
Booking flow	booking success rate, seat contention rate, booking latency
Partner operations	theatre onboarding SLA, show publish success rate
Business	conversion rate, seat occupancy, average booking value, platform fee revenue

6. Test Scenarios and Curl Commands

> These examples use `curl.exe` with PowerShell line continuation. If your database already has data, replace `movieId`, `theatreId`, `showId`, and `bookingId` with the actual values returned by your environment.

6.1 Base Setup for an Offer Location

This sequence creates an eligible location for the third-ticket offer: `Mumbai` + `PVR Andheri`.

```
# 1. Onboard theatre partner
curl.exe -X POST "http://localhost:8080/partners/theatres" `
  -H "Content-Type: application/json" `
  -d '{"theatreName":"PVR Andheri","cityName":"Mumbai"}'

# 2. Create a movie
curl.exe -X POST "http://localhost:8080/movies" `
  -H "Content-Type: application/json" `
  -d '{"title":"Interstellar","genre":"Sci-Fi","language":"English"}'

# 3. Create an afternoon show in the offer location
curl.exe -X POST "http://localhost:8080/partners/shows" `
  -H "Content-Type: application/json" `
  -d '{"movieId":1,"theatreId":1,"showDate":"2026-04-04","showTime":"13:00","price":100.0}'
```

```
# 4. Allocate seat inventory
curl.exe -X POST "http://localhost:8080/partners/shows/1/seats" `
  -H "Content-Type: application/json" `
  -d '{"seatNumbers":["A1","A2","A3","A4","A5"]}'
```

6.2 Read Scenario: Browse Theatres Running the Show by City and Date

This validates the read scenario from the exercise.

```
curl.exe "http://localhost:8080/browse/shows?movieId=1&city=Mumbai&date=2026-04-04"
```

Expected result: a non-empty array with the configured show, theatre details, show date, show time, and price.

6.3 Write Scenario: Book Tickets for an Eligible Afternoon Offer Show

This validates both pricing rules together:

- third-ticket 50% off because the show is in an eligible city/theatre
- 20% afternoon discount because the show time is `13:00`

```
curl.exe -X POST "http://localhost:8080/bookings" `
  -H "Content-Type: application/json" `
  -d '{"showId":1,"seats":["A1","A2","A3"]}'
```

Expected price calculation:

```
3 x 100 = 300
third ticket 50% off ⇒ 300 - 50 = 250
afternoon 20% off   ⇒ 250 x 0.8 = 200
```

Expected response field: `"totalPrice": 200.0`

6.4 Alternate Pricing Scenario: Afternoon Discount Only

This sequence uses a non-eligible theatre and city so only the afternoon discount should apply.

```
# 1. Onboard non-offer theatre
curl.exe -X POST "http://localhost:8080/partners/theatres" `
  -H "Content-Type: application/json" `
  -d '{"theatreName":"Downtown Screens","cityName":"Delhi"}'

# 2. Create a second afternoon show in a non-offer location
curl.exe -X POST "http://localhost:8080/partners/shows" `
  -H "Content-Type: application/json" `
  -d '{"movieId":1,"theatreId":2,"showDate":"2026-04-04","showTime":"13:00","price":100.0}'

# 3. Allocate seats
curl.exe -X POST "http://localhost:8080/partners/shows/2/seats" `
  -H "Content-Type: application/json" `
  -d '{"seatNumbers":["B1","B2","B3","B4"]}'

# 4. Book three seats
curl.exe -X POST "http://localhost:8080/bookings" `
  -H "Content-Type: application/json" `
  -d '{"showId":2,"seats":["B1","B2","B3"]}'
```

Expected price calculation:

```
3 x 100 = 300
third ticket discount ⇒ not applicable
afternoon 20% off     ⇒ 300 x 0.8 = 240
```

Expected response field: `"totalPrice": 240.0`

6.5 Alternate Pricing Scenario: Third-Ticket Discount Only

This sequence uses an eligible city/theatre but an evening show, so only the third-ticket offer applies.

```
# 1. Create an evening show at the offer theatre
curl.exe -X POST "http://localhost:8080/partners/shows" `
  -H "Content-Type: application/json" `
  -d '{"movieId":1,"theatreId":1,"showDate":"2026-04-05","showTime":"18:30","price":100.0}'

# 2. Allocate seats
curl.exe -X POST "http://localhost:8080/partners/shows/3/seats" `
  -H "Content-Type: application/json" `
  -d '{"seatNumbers":["C1","C2","C3","C4"]}'

# 3. Book three seats
curl.exe -X POST "http://localhost:8080/bookings" `
  -H "Content-Type: application/json" `
  -d '{"showId":3,"seats":["C1","C2","C3"]}'
```

Expected price calculation:

```
3 x 100 = 300
third ticket 50% off ⇒ 300 - 50 = 250
afternoon discount   ⇒ not applicable
```

Expected response field: `"totalPrice": 250.0`

6.6 Another Movie Scenario

This validates catalog flexibility for a second movie while reusing the same theatre partner.

```
# 1. Create another movie
curl.exe -X POST "http://localhost:8080/movies" `
  -H "Content-Type: application/json" `
  -d '{"title":"Inception","genre":"Thriller","language":"English"}'

# 2. Create an afternoon show
curl.exe -X POST "http://localhost:8080/partners/shows" `
  -H "Content-Type: application/json" `
  -d '{"movieId":2,"theatreId":1,"showDate":"2026-04-06","showTime":"15:00","price":120.0}'

# 3. Allocate seats
curl.exe -X POST "http://localhost:8080/partners/shows/4/seats" `
  -H "Content-Type: application/json" `
  -d '{"seatNumbers":["D1","D2","D3","D4","D5"]}'

# 4. Browse this movie
curl.exe "http://localhost:8080/browse/shows?movieId=2&city=Mumbai&date=2026-04-06"

# 5. Book three seats
curl.exe -X POST "http://localhost:8080/bookings" `
  -H "Content-Type: application/json" `
  -d '{"showId":4,"seats":["D1","D2","D3"]}'
```

Expected price calculation:

```
3 x 120 = 360
third ticket 50% off ⇒ 360 - 60 = 300
afternoon 20% off    ⇒ 300 x 0.8 = 240
```

Expected response field: `"totalPrice": 240.0`

6.7 Fetch a Persisted Booking

```
curl.exe "http://localhost:8080/bookings/1"
```

7. Suggested Discussion Points for the Exercise

- **Why this scope:** it directly demonstrates API contract, service logic, data model, design decisions, and curl-verifiable scenarios.
- **Why Ehcache now:** current local single-service deployment benefits from low-latency in-process caching without extra operational complexity.
- **How to extend:** add partner show update/delete, bulk booking/cancellation, payment workflow, and distributed cache if horizontal scale is required.
- **How to productionize:** add authn/authz, idempotency, optimistic locking, observability, CI/CD gates, and stronger test automation.

8. Deliverable Summary

This documentation is designed to support the technical discussion for the attached exercise by showing:

- the implemented functional scope
- the architecture and service flows
- the key design and non-functional reasoning
- curl commands to demonstrate the scenarios from the exercise in a running environment